Bài thực hành 3

Rẽ nhánh và vòng lặp trên dữ liệu thật

Hà Minh Tuấn

Khoa Toán — Tin học, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

Phần A: 2,5 tiết tại lớp · Phần B: làm ở nhà

Mục tiêu

- Viết điều kiện `if`/`elif` và `match-case` đúng ý, tránh được ba lỗi điều kiện không báo lỗi của Bài 4.
- Dùng `for` với `enumerate`, `zip`, `.items()`; dùng `while`, `break`, `continue` đúng chỗ.
- Dùng các phương thức thông dụng của `list`, `tuple`, `dict`, `set` khi tiền xử lý: sắp xếp, đếm, lấy giá trị khác nhau, phép toán tập hợp, gom nhóm.
- Áp dụng năm mẫu lặp — đếm, tích luỹ, lọc, cực trị, gom nhóm — trên một bộ dữ liệu 1000 dòng.
- Đọc và làm sạch một tệp dữ liệu có lỗi nhập liệu: chuẩn hoá giá trị, loại dòng hỏng, bỏ bản ghi trùng.

Bài này gồm hai phần

Phần	Làm ở đâu	Cách viết
A. Phân tích điểm thi của 1000 học sinh	Tại lớp, trên máy của Trường. 2,5 tiết.	Gõ mã mẫu, rồi viết các hàm tương tự trong `tai_lop.py`; mỗi mục có kết quả để đối chiếu.
B. Làm sạch bản dữ liệu nhập tay	Ở nhà, tự làm. **Có tính điểm.**	Chỉ nêu *yêu cầu* và vài gợi ý, không hướng dẫn từng bước.

Chuẩn bị

Mở sẵn slide **Bài 4 — Rẽ nhánh và vòng lặp** để tra cứu.

Tải từ Classroom về ba tệp: `tai_lop.py`, `StudentsPerformance.csv` và `hoc_sinh_ban.csv`.

Mục lục

Mục tiêu **1**

PHẦN A — LÀM TẠI LỚP **3**

1 Mở dự án và đọc dữ liệu (10 phút) **3**

1.1 Dữ liệu 3

2 Rẽ nhánh và `match-case` (20 phút) **4**
2.1 Mã mẫu: `can_ho_tro(hs)` . . . 4
2.2 Sinh viên tự thực hiện . . . 4

3 Phương thức của `list`, `tuple`, `dict`, `set` (35 phút) **5**
3.1 Các phương thức thường dùng . . . 5
3.2 Mã mẫu: `mau_phuong_thuc(ds_hs)` . . . 5
3.3 Sinh viên tự thực hiện . . . 6

4 Năm mẫu lặp trên 1000 học sinh (35 phút) **7**
4.1 Mã mẫu: `tom_tat_mon(ds_hs, mon)` . . . 7
4.2 Sinh viên tự thực hiện . . . 7

5 `while` và `break` (20 phút) **8**
5.1 Mã mẫu: `vi_tri_dau_tien(ds_hs, dieu_kien)` . . . 8
5.2 Sinh viên tự thực hiện . . . 8

Hết phần A — kiểm lại trước khi rời phòng máy **9**

PHẦN B — LÀM Ở NHÀ **10**

6 Yêu cầu **10**
6.1 Dữ liệu vào . . . 10
6.2 Năm bước đã chia sẵn . . . 11
6.3 Hai hàm mở rộng . . . 11

7 Gợi ý **12**

8 Nộp bài **13**

PHẦN A — LÀM TẠI LỚP

Mục	Nội dung	Thời lượng
A1	Mở dự án và đọc dữ liệu	10 phút
A2	Rẽ nhánh và `match-case`	20 phút
A3	Phương thức của `list`, `tuple`, `dict`, `set`	35 phút
A4	Năm mẫu lặp trên 1000 học sinh	35 phút
A5	`while` và `break`	20 phút
	Tổng	**120 phút**

Cả phần A làm trong tệp `tai_lop.py`. Từ A2 đến A5, mỗi mục gồm hai bước:

1. **Mã mẫu**: gõ lại thân hàm in trong đề vào đúng hàm có chú thích `# <-- MA MAU` trong `tai_lop.py`, chạy, đối chiếu kết quả.
2. **Sinh viên tự thực hiện**: viết các hàm có chú thích `# <-- CAN VIET LAI THAN HAM`, cùng kiểu với mã mẫu nhưng trả lời một câu hỏi khác.

Mỗi lần sửa xong một hàm thì chạy `python tai_lop.py` và đối chiếu với hộp **Mốc kiểm** của mục đó. Chưa đạt thì sửa lại hoặc hỏi giảng viên hướng dẫn trước khi làm mục sau.

1. Mở dự án và đọc dữ liệu (10 phút)

Mục đích

Tạo dự án tuần 3 và nạp bộ dữ liệu điểm thi của 1000 học sinh.

1. Trong VS Code: **File** → **Open Folder...**, tạo thư mục mới `python-tuan03` trên ổ `D:\`, rồi **Select Folder**.
2. Kéo ba tệp đã tải vào khung *Explorer*.
3. **Terminal** → **New Terminal**, tạo và kích hoạt môi trường ảo, rồi chạy thử:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python tai_lop.py
```

1.1 Dữ liệu

`StudentsPerformance.csv` là bộ dữ liệu *Students Performance in Exams* trên Kaggle: mỗi dòng là một học sinh, tám cột. Hàm `doc_du_lieu` trong `tai_lop.py` đã viết sẵn, trả về một `list` gồm 1000 `dict`, mỗi `dict` có tám khoá:

Cột trong tệp	Khoá	Giá trị
gender	gioi_tinh	female, male
race/ethnicity	nhom	group A … group E
parental level of education	hoc_van	sáu mức, từ some high school tới master's degree
lunch	an_trua	standard, free/reduced
test preparation course	on_thi	none, completed
math/reading/writing score	toan, doc, viet	số nguyên từ 0 đến 100

Mốc kiểm

`python tai_lop.py` in dòng đầu tiên là `So hoc sinh: 1000`. Các dòng sau là giá trị rỗng hoặc bằng 0, vì các hàm chưa được viết.

2. Rẽ nhánh và `match-case` (20 phút)

Mục đích

Viết các hàm phân loại một học sinh bằng `if`/`elif`, toán tử `and`/`or` và `match-case`.

2.1 Mã mẫu: `can_ho_tro(hs)`

Một học sinh cần hỗ trợ nếu ăn trưa `"free/reduced"` **và** không ôn thi; **hoặc** có ít nhất một môn dưới 40 điểm. Mỗi vế `or` là một phép so sánh đầy đủ.

tai_lop.py — mã mẫu A2

```
def can_ho_tro(hs):
    kho_khan = hs["an_trua"] == "free/reduced" and hs["on_thi"] == "none"
    co_mon_yeu = hs["toan"] < 40 or hs["doc"] < 40 or hs["viet"] < 40
    return kho_khan or co_mon_yeu
```

Slide Bài 4: *Ghép điều kiện*; *Cạm bẫy 2: viết `or` theo lối nói tiếng Việt.*

2.2 Sinh viên tự thực hiện

- `xep_loai(diem)`: trả về `"A"` nếu điểm từ 90, `"B"` từ 80, `"C"` từ 70, `"D"` từ 50, còn lại `"F"`. Slide Bài 4: *Cạm bẫy 1: thứ tự `elif` quyết định kết quả.*
- `bac_hoc_van(muc)`: viết bằng `match-case`, đổi trình độ học vấn của phụ huynh sang bậc theo bảng dưới. Slide Bài 4: *Ba dạng mẫu dùng nhiều nhất*, dạng 1.

Giá trị	Bậc
some high school, high school	1
some college, associate's degree	2
bachelor's degree	3
master's degree	4
giá trị khác	None

Phần `--- A2` phải in ra đúng như sau

```
--- A2
True False
['A', 'A', 'B', 'C', 'D', 'F', 'F']
[1, 4, None]
```

Dòng thứ hai ra `['D', 'D', 'D', 'D', 'D', 'F', 'F']`: điều kiện lỏng nhất (`>= 50`) đang nằm trên cùng.

3. Phương thức của `list`, `tuple`, `dict`, `set` (35 phút)

Mục đích

Dùng các phương thức có sẵn của bốn kiểu dữ liệu để làm những việc lặp đi lặp lại khi tiền xử lý: lấy một cột ra danh sách, sắp xếp, đếm, lấy giá trị khác nhau, so sánh hai nhóm, gom bản ghi theo nhóm.

3.1 Các phương thức thường dùng

Kiểu	Phương thức	Dùng để
`list`	`append(x)`, `extend(ds)`	thêm một phần tử; nối thêm cả danh sách
	`count(x)`, `index(x)`	đếm số lần xuất hiện; vị trí lần xuất hiện đầu
	`sort()`, `sort(reverse=True)`	sắp xếp *tại chỗ*, trả về `None`
	`sorted(ds)`	trả về danh sách *mới* đã sắp, không đổi `ds`
	`pop()`, `remove(x)`	lấy ra phần tử cuối; xoá lần xuất hiện đầu của `x`
`tuple`	`a, b = t`	tách bản ghi thành từng biến
	dùng làm khoá của `dict`/`set`	khoá ghép từ nhiều cột, ví dụ `(gioi_tinh, an_trua)`
`dict`	`get(k, mac_dinh)`	đọc khoá có thể chưa có, dùng để đếm
	`setdefault(k, [])`	tạo danh sách rỗng cho khoá mới, dùng để gom nhóm
	`items()`, `keys()`, `values()`	duyệt cặp khoá–giá trị, khoá, giá trị
	`update(d2)`, `pop(k, None)`	gộp thêm từ điển khác; bỏ một khoá
`set`	`set(ds)`, `add(x)`	lấy các giá trị khác nhau; thêm một phần tử
	`a & b`, `a | b`, `a - b`	giao, hợp, hiệu của hai tập
	`x in s`	kiểm tra có mặt, nhanh hơn nhiều so với `x in list`

3.2 Mã mẫu: `mau_phuong_thuc(ds_hs)`

Hàm này không trả về gì; nó in kết quả của từng nhóm phương thức để quan sát.

tai_lop.py — mã mẫu A3

```python
def mau_phuong_thuc(ds_hs):
    # list: append, count, index, sort
    diem_toan = []
```

```
    for hs in ds_hs:
        diem_toan.append(hs["toan"])
    print(len(diem_toan), diem_toan.count(100), diem_toan.index(100) + 1)
    diem_toan.sort()
    print(diem_toan[:5], diem_toan[-3:])

    # set: add, cac phep toan tap hop
    nu, dat_toan = set(), set()
    for stt, hs in enumerate(ds_hs, start=1):
        if hs["gioi_tinh"] == "female":
            nu.add(stt)
        if hs["toan"] >= 50:
            dat_toan.add(stt)
    print(len(nu & dat_toan), len(nu - dat_toan), len(nu | dat_toan))

    # tuple lam khoa cua dict; dict: get, items
    dem = {}
    for hs in ds_hs:
        khoa = (hs["gioi_tinh"], hs["an_trua"])
        dem[khoa] = dem.get(khoa, 0) + 1
    for khoa, n in sorted(dem.items()):
        print(khoa, n)

    # dict: setdefault de gom danh sach
    toan_theo_nhom = {}
    for hs in ds_hs:
        toan_theo_nhom.setdefault(hs["nhom"], []).append(hs["toan"])
    nhom_a = toan_theo_nhom["group A"]
    print(sorted(toan_theo_nhom), len(nhom_a), max(nhom_a))
```

Phần `--- A3 mau` phải in ra đúng như sau

```
--- A3 mau
1000 7 150
[0, 8, 18, 19, 22] [100, 100, 100]
432 86 951
('female', 'free/reduced') 189
('female', 'standard') 329
('male', 'free/reduced') 166
('male', 'standard') 316
['group A', 'group B', 'group C', 'group D', 'group E'] 89 100
```

Đọc kết quả: 7 học sinh được 100 điểm toán, người đầu tiên là học sinh thứ 150; 432 học sinh nữ đạt môn toán, 86 học sinh nữ không đạt; nhóm `group A` có 89 học sinh.

3.3 Sinh viên tự thực hiện

- `top_k(ds_hs, mon, k)`: danh sách `k` điểm cao nhất của môn `mon`, sắp giảm dần.
- `gioi_ca_ba(ds_hs, nguong)`: trả về cặp hai `set` số thứ tự (đếm từ 1): học sinh có cả ba môn từ `nguong` trở lên; và học sinh *chỉ* môn toán từ `nguong` trở lên, đọc và viết đều dưới `nguong`.

- `gom_theo(ds_hs, cot)`: `dict`, mỗi giá trị của cột `cot` → danh sách số thứ tự (đếm từ 1) các học sinh có giá trị đó, theo thứ tự trong dữ liệu. Dùng `setdefault`.

Phần `---` `A3` phải in ra đúng như sau

```
--- A3
[100, 100, 100, 100, 100, 100, 100, 99, 99, 99] []
28 [3, 115, 150, 166, 180] 20
['completed', 'none'] 358 [2, 7, 9, 14, 19]
```

- Dòng đầu ra `[0, 8, 18, ...]`: thiếu `reverse=True`.
- Dừng với `TypeError: 'NoneType' object is not subscriptable`: đã viết `diem = diem.sort()`. `sort()` sắp tại chỗ và trả về `None`.
- Số cuối dòng hai ra 98: đã viết `toan - doc | viet`. Phép `-` được tính trước phép `|`, nên cần ngoặc.
- Dòng ba ra `[1, 6, 8, 13, 18]`: `enumerate` đang đếm từ 0.

4. Năm mẫu lặp trên 1000 học sinh (35 phút)

Mục đích

Trả lời các câu hỏi thống kê về bộ dữ liệu bằng năm mẫu lặp của Bài 4.

4.1 Mã mẫu: `tom_tat_mon(ds_hs, mon)`

Trả về cặp (*số học sinh đạt*, *điểm trung bình*) của một môn; đạt là từ 50 điểm. Một vòng lặp làm cùng lúc hai mẫu: đếm và tích luỹ. Cả hai biến khởi tạo *trước* vòng lặp.

tai_lop.py — mã mẫu A4

```python
def tom_tat_mon(ds_hs, mon):
    dat, tong = 0, 0
    for hs in ds_hs:
        tong += hs[mon]
        if hs[mon] >= 50:
            dat += 1
    return (dat, round(tong / len(ds_hs), 2))
```

Slide Bài 4: *Năm mẫu lặp trong xử lý dữ liệu*; *Từ lời phát biểu đến vòng lặp*.

4.2 Sinh viên tự thực hiện

- `cao_nhat(ds_hs, mon)`: cặp (*số thứ tự*, *điểm*) của học sinh có điểm môn `mon` cao nhất. Số thứ tự đếm từ 1; nhiều học sinh bằng điểm thì lấy người đứng trước. Mẫu cực trị kèm nhãn, dùng `enumerate`.
- `tb_theo_nhom(ds_hs, cot, mon)`: `dict`, mỗi giá trị của cột `cot` → điểm trung bình môn `mon` của các học sinh có giá trị đó, làm tròn 2 chữ số. Mẫu gom nhóm kết hợp tích luỹ: hai từ điển tổng và đếm, rồi duyệt bằng `.items()`.

Phần `--- A4` phải in ra đúng như sau

```
--- A4
(865, 66.09)
toan (150, 100)
doc (107, 100)
viet (107, 100)
{'none': 64.08, 'completed': 69.7}
{"bachelor's degree": 73.38, 'some college': 68.84,
 "master's degree": 75.68, "associate's degree": 69.9,
 'high school': 62.45, 'some high school': 64.89}
```

(Dòng cuối trên màn hình là một dòng dài; ở đây ngắt làm ba cho vừa trang.)

- Dòng `toan` ra `(963, 100)`: đã dùng `>=`, nên người đứng *sau* ghi đè người đứng trước.
- Dòng `toan` ra `(149, 100)`: `enumerate` đang đếm từ 0.
- Dòng `on_thi` ra `{'none': 77.0, 'completed': 68.0}`: hai từ điển tổng và đếm bị khởi tạo lại ở mỗi vòng lặp, nên mỗi nhóm chỉ còn điểm của học sinh cuối cùng.

5. `while` và `break` (20 phút)

Mục đích

Viết vòng lặp dừng ngay khi tìm thấy kết quả, và vòng lặp chạy đến khi đạt điều kiện mà không biết trước số lần.

5.1 Mã mẫu: `vi_tri_dau_tien(ds_hs, dieu_kien)`

Trả về số thứ tự của học sinh *đầu tiên* có `can_ho_tro(hs) == dieu_kien`; không có thì trả về `None`. `break` dừng vòng lặp ngay khi tìm thấy.

tai_lop.py — mã mẫu A5

```python
def vi_tri_dau_tien(ds_hs, dieu_kien):
    vi_tri = None
    for stt, hs in enumerate(ds_hs, start=1):
        if can_ho_tro(hs) == dieu_kien:
            vi_tri = stt
            break
    return vi_tri
```

Slide Bài 4: *`break` thoát hẳn, `continue` bỏ qua một vòng*; *`while` — lặp đến khi điều kiện sai*; *Vòng lặp vô hạn*.

5.2 Sinh viên tự thực hiện

`diem_cong_can_thiet(ds_hs, mon, ti_le)`: nhà trường cộng đều cho mọi học sinh cùng một số điểm $k = 0, 1, 2, \ldots$ sao cho tỉ lệ học sinh đạt môn `mon` (điểm $+ k \geq 50$) không thấp hơn `ti_le`. Trả về k nhỏ nhất. Số lần lặp không biết trước: dùng `while`.

Phần `--- A5` phải in ra đúng như sau

```
--- A5
4 1
9
```

- Dòng hai ra `10`: điều kiện dừng dùng `>` thay cho `>=`. Với $k = 9$ tỉ lệ đạt đúng bằng `0.95`.
- Dòng hai không in ra và chương trình không dừng: biến k không được tăng trong vòng lặp. Nhấn `Ctrl+C` để dừng.

Hết phần A — kiểm lại trước khi rời phòng máy

Ba điều phải đúng

1. `python tai_lop.py` in ra đúng các phần từ `--- A2` đến `--- A5`.
2. `bac_hoc_van` viết bằng `match-case`, `gom_theo` dùng `setdefault`, `diem_cong_can_thiet` viết bằng `while`.
3. Không hàm nào khởi tạo biến đếm hay biến tổng bên trong vòng lặp.

Giữ lại bài làm

Chép `tai_lop.py` ra USB hoặc lên Google Drive trước khi ra về. Phần B dùng lại các hàm trong tệp này.

PHẦN B — LÀM Ở NHÀ

Phần này **có tính điểm**.

6. Yêu cầu

Yêu cầu lớn

Tệp `hoc_sinh_ban.csv` chứa cùng bộ dữ liệu ở phần A nhưng được nhập lại bằng tay, có lỗi nhập liệu. Viết chương trình đọc tệp, loại các dòng hỏng, chuẩn hoá giá trị, bỏ bản ghi trùng, rồi thống kê điểm trung bình theo nhóm và vẽ biểu đồ tần suất điểm bằng ký tự.

Viết một tệp `<MSSV>_lab03.py` chứa **bảy hàm** dưới đây. Lấy tệp `MAU_BAI_NOP.py` trên Classroom làm khung. Năm hàm đầu là các bước đã chia sẵn; hai hàm cuối là phần mở rộng.

6.1 Dữ liệu vào

Tệp có 1018 dòng, dòng đầu là tiêu đề, cùng tám cột như phần A. Các lỗi có trong tệp:

Lỗi	Ví dụ
Giới tính ghi nhiều kiểu	`F`, `Female`, `nu`, `MALE`, `nam`; vài dòng ghi `?` hoặc `unknown`
Cột phân loại sai hoa thường, thừa dấu cách	`GROUP C`, `Some College`, `" standard"`
Thiếu điểm	ô trống, `NA`, `n/a`, `-`
Điểm ngoài khoảng 0–100	`105`, `-5`, `999`
Điểm ghi dạng số thực	`72.0`
Sai số trường	dòng thiếu một trường hoặc thừa một trường
Dòng trống, dòng trùng	dòng chỉ có dấu cách; cùng một dòng xuất hiện hai lần

Các hàm nhận dữ liệu dưới dạng `list` các dòng chữ. Đọc tệp thành danh sách dòng như sau:

```
with open("hoc_sinh_ban.csv", encoding="utf-8") as tep:
    cac_dong = tep.readlines()
```

Bài làm được chạy với dữ liệu khác

Các hàm sẽ được chạy với dữ liệu khác, gồm cả danh sách rỗng, học sinh có điểm 0, nhóm không có học sinh nào có điểm, và các cách viết khác của cùng một lỗi trong bảng trên.

Dữ liệu kiểm tra luôn bảo đảm

- Ô điểm, nếu không thuộc các kiểu *thiếu điểm* ở bảng trên, luôn là một số có giá trị nguyên, có thể viết kèm `.0`.
- Dòng tiêu đề là dòng bắt đầu bằng `gender`.

6.2 Năm bước đã chia sẵn

Giữ **đúng tên hàm và đúng thứ tự tham số**. Sai tên thì hàm không được tìm thấy, và bài coi như bỏ trống dù thuật toán viết đúng.

Hàm	Trả về
`chuan_hoa_gioi_tinh(s)`	`"female"` với `female`, `f`, `nu`; `"male"` với `male`, `m`, `nam` — không phân biệt hoa thường, bỏ dấu cách thừa. Giá trị khác → `None`. Viết bằng `match-case`.
`doc_diem(s)`	số nguyên từ 0 đến 100. Thiếu điểm hoặc ngoài khoảng → `None`. `"72.0"` → 72.
`doc_ban_ghi(dong)`	`dict` tám khoá như phần A. Giới tính qua `chuan_hoa_gioi_tinh`; bốn cột phân loại còn lại bỏ dấu cách thừa và đổi sang chữ thường; ba điểm qua `doc_diem` (có thể là `None`). Trả về `None` nếu dòng không có đúng 8 trường hoặc giới tính là `None`.
`doc_tat_ca(cac_dong)`	cặp (*danh sách học sinh hợp lệ*, *số dòng hỏng*). Bỏ qua dòng tiêu đề và dòng trống, không tính chúng là dòng hỏng.
`tb_theo_nhom(ds_hs, cot, mon)`	như ở A4, nhưng bỏ qua học sinh có điểm môn `mon` là `None`. Nhóm không có học sinh nào có điểm thì không có mặt.

Hàm	Ở lớp đã làm tại
`chuan_hoa_gioi_tinh`	A2 `bac_hoc_van` (`case ... | ...`)
`doc_diem`	A2 `xep_loai`; slide Bài 4 *Python cho phép viết nối chuỗi như trong toán*
`doc_ban_ghi`	slide Bài 4 *Ba dạng mẫu*, dạng 3 (mẫu cấu trúc)
`doc_tat_ca`	A4 mẫu đếm; slide Bài 4 `continue`
`tb_theo_nhom`	A4 `tb_theo_nhom`; A3 `setdefault`

Với hoc_sinh_ban.csv

```
>>> ds, so_loi = doc_tat_ca(cac_dong)
>>> len(ds), so_loi
(1002, 10)
```

6.3 Hai hàm mở rộng

Phần này *chưa hướng dẫn ở lớp*.

`bo_trung_lap(ds_hs)`

Trả về cặp (*danh sách mới*, *số bản ghi bị bỏ*). Hai học sinh trùng nhau khi hai `dict` bằng nhau. Giữ lần xuất hiện đầu tiên, giữ nguyên thứ tự. Danh sách `ds_hs` không được thay đổi. Với kết quả `doc_tat_ca` của `hoc_sinh_ban.csv`: còn 990 học sinh, bỏ 12. Sau khi bỏ trùng, `tb_theo_nhom(ds, "gioi_tinh", "toan")` cho `{'female': 63.54, 'male': 68.87}`.

`bieu_do_diem(ds_hs, mon, do_rong)`

Trả về danh sách các dòng chữ của biểu đồ tần suất điểm môn `mon`. `do_rong` là ước của 100 (5, 10, 20, 25, 50).

- Chia đoạn 0–100 thành `100 // do_rong` khoảng: `0` đến `do_rong - 1`, tiếp theo, …; khoảng cuối chứa luôn điểm 100.
- Mỗi khoảng một dòng: `f"{a:>3}-{b:>3} | {'#' * so_dau} {n}"`, với `a`, `b` là hai đầu khoảng, `n` là số học sinh trong khoảng, `so_dau = n * 40 // n_lon_nhat`.
- Bỏ qua điểm `None`. Không có điểm nào thì trả về danh sách rỗng.

Ví dụ: in từng dòng của bieu_do_diem(ds, "toan", 10), ds đã bỏ trùng

```
 0-  9 |  2
10- 19 |  2
20- 29 | # 10
30- 39 | ### 25
40- 49 | ############## 93
50- 59 | ########################## 182
60- 69 | ######################################## 264
70- 79 | ############################## 210
80- 89 | ################### 133
90-100 | ######## 57
```

Hai quy định về cách chia bước

1. `doc_ban_ghi` phải gọi `chuan_hoa_gioi_tinh` và `doc_diem`.
2. **Không hàm nào dài quá 25 dòng**, tính cả dòng `def` và chú thích.

Được viết thêm hàm phụ ngoài bảy hàm đã nêu tên.

7. Gợi ý

Cho năm hàm đầu

1. `chuan_hoa_gioi_tinh`: chuẩn hoá chuỗi trước, rồi mới đưa vào `match`.
2. `doc_diem`: `int("72.0")` báo `ValueError` (slide Bài 3, *Khi ép kiểu thất bại*).
3. `doc_ban_ghi`: `match dong.split(",")` với một `case` liệt kê đúng tám tên vừa kiểm tra số trường vừa đặt tên cho từng trường.
4. `doc_tat_ca`: dòng nào bị bỏ qua và dòng nào bị tính là hỏng là hai việc khác nhau.

Cho phần mở rộng

`bo_trung_lap`: `dict` không bỏ được vào `set`, nhưng `tuple` thì được. Đổi mỗi học sinh thành `tuple` các giá trị và dùng một `set` để nhớ những bản ghi đã gặp (A3).
`bieu_do_diem`: tách việc *đếm số học sinh trong từng khoảng* ra một hàm riêng, trả về danh sách số đếm; hàm vẽ chỉ còn việc định dạng.

Lỗi không có thông báo

Trong `tb_theo_nhom`, viết `if hs[mon]:` để bỏ qua điểm thiếu thì điểm `0` cũng bị bỏ qua, vì `0` được coi là sai. Tệp có một học sinh được 0 điểm toán: kết quả nữ đổi từ `63.54` thành `63.67`. Phải viết `if hs[mon] is not None:` (slide Bài 4, *Cạm bẫy 3*).

8. Nộp bài

Ghi nhớ

Nộp **một tệp duy nhất** tên `<MSSV>_lab03.py` lên Classroom. Ví dụ sinh viên 24280001 nộp tệp `24280001_lab03.py`. Số bài ghi hai chữ số.
Trong tệp chứa bảy hàm theo đúng tên đã cho, các hàm phụ tự viết, và phần tự thử nghiệm — nhưng phần thử nghiệm, kể cả lệnh mở tệp `hoc_sinh_ban.csv`, phải nằm trong khối `if __name__ == "__main__":` ở cuối tệp, nếu không nó sẽ chạy mỗi khi tệp được mở ra chấm.
Không dùng `import` nào.

Hạn nộp

24 giờ trước giờ bắt đầu ca thực hành tiếp theo của lớp.
Ví dụ ca thực hành 4 của lớp bắt đầu lúc 7g30 thứ Năm thì hạn nộp là 7g30 thứ Tư.